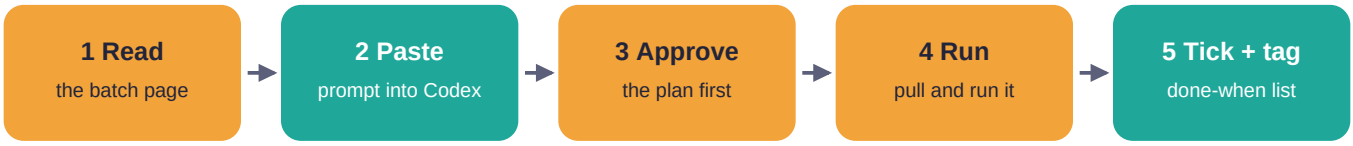
Learning OS

Twelve batches. You run every step, see every result, and finish with a product you built yourself.

What this is: a step-by-step build plan, from an empty repository to a finished demo. Every batch opens with a diagram of what you are about to build, then tells you what to do, what to paste into Codex, what output to expect, and how to know it worked before you move on.

The rhythm of every batch



Codex works in the cloud and opens a pull request for each batch. You read the plan before it writes code, then you merge, pull the code to your own machine and run it yourself. Where a step says run something in your terminal and you only have a cloud task, ask Codex to run it and show you the raw output; you still read it and judge it.

How to read the diagrams

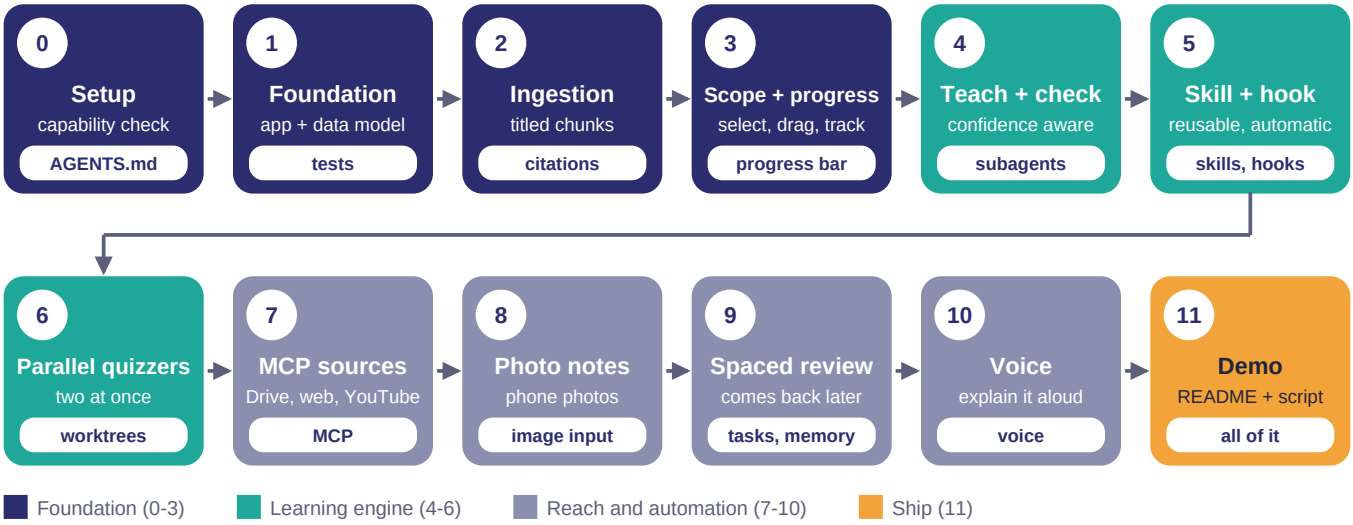
Each batch page starts with a strip showing how far along the journey you are, then a flow of boxes. Teal means built in this batch. Grey means reused from an earlier batch. Amber is you, the learner or the builder. Indigo is something external or automatic, such as Drive or a schedule.

Four rules that keep you in control

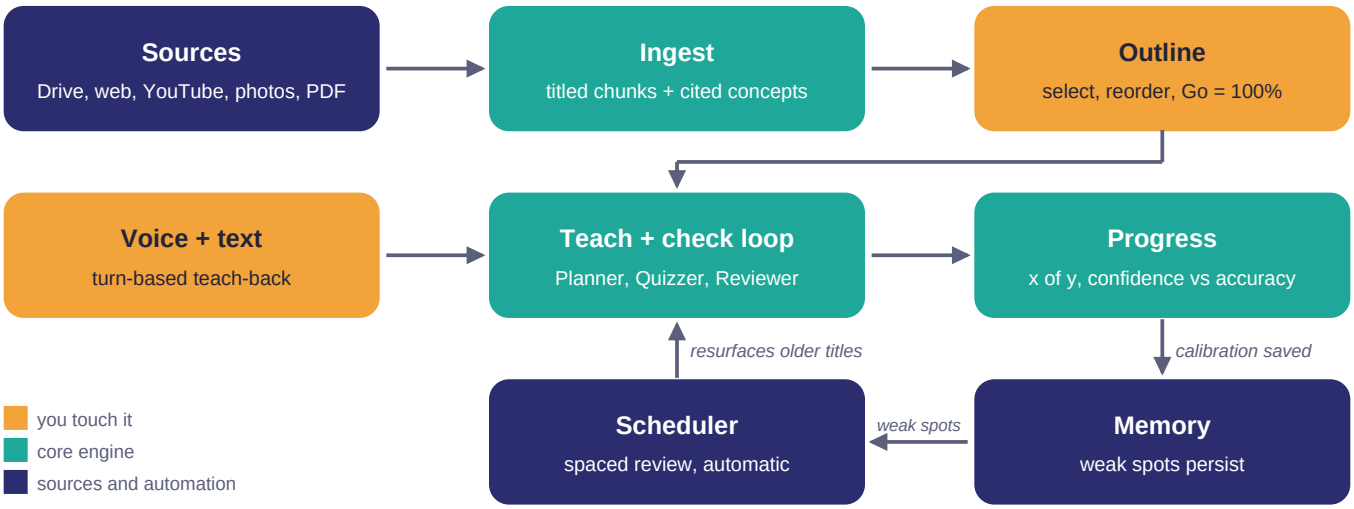
- Never approve a plan you have not read. If it does not match the prompt, say so and ask again.
- Never skip a done-when item. If one fails, paste the error to Codex and ask it to explain the cause before fixing it.
- Tag every finished batch. If a later batch breaks things, git reset --hard batch-N-done takes you back to a known good point.
- Only claim in your demo what you have seen run. docs/capabilities.md is your source of truth.

i The whole picture

The journey: twelve batches in four phases



What the finished product looks like



Sources feed one ingestion pipeline. You choose a scope in the outline. The teach-and-check loop works through that scope by text or voice, and progress is measured against it. Calibration history is kept, and the scheduler uses it to bring weak titles back.

ii Decisions and roadmap

Defaults locked for the build

These are my suggested defaults. Change any of them now, before Batch 1, and add the change to AGENTS.md so Codex follows it.

Decision	Default	Why
Stack	Python 3.12, FastAPI, SQLite, vanilla JavaScript with SortableJS	Small, fast to read, easy for you to inspect.
Model calls	One file, app/llm.py, using the OpenAI API	One place to swap or stub the model.
Collections	A database row, optionally tied to one Drive folder	Matches the design document.
Scope and progress	Selected titles are the 100 percent; the Reviewer judges completion	Your agreed design.
Out of scope	Gated third-party courses, video file upload	Not built, so never claimed.

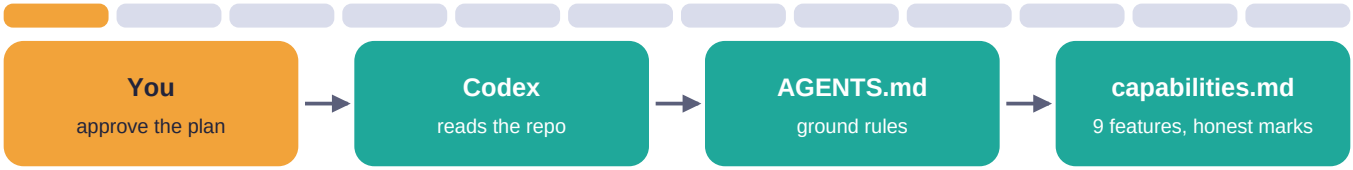
Roadmap: what you will see at the end of each batch

#	Batch	You will see	Codex feature
0	Setup and capability check	Codex in your repo, plus an honest feature map	AGENTS.md, config
1	Foundation: app and data model	The app boots and tests pass	plan and tests
2	Ingestion: titled chunks with citations	Chapters printed as titled sections with citations	cloud task, stubs
3	Scope and progress: choose, drag, track	Select titles, drag to reorder, watch the bar	UI review
4	Teach and check, with subagents	A confidence-aware question and verdict	subagents
5	A reusable skill and an automatic hook	A skill file and a log entry that appears by itself	skills, hooks
6	Parallel quizzers in git worktrees	Two quiz sets made in two worktrees	worktrees
7	MCP: Drive folder, web page and YouTube	A Drive file and a link appear in the outline	MCP
8	Photo notes	Notes photo turned into concepts	image input
9	Spaced review and memory	Due titles after a date jump	scheduled tasks, memory
10	Voice teach-back	A three-turn spoken exchange	voice
11	Polish and demo	A clean clone that runs, and a rehearsed demo	all

0 Setup and capability check

Codex feature on show: AGENTS.md and config, plus an honest map of the nine Codex features

YOU ARE HERE: BATCH 0 OF 11



You steer. Codex writes two files that govern every batch after this one.

new in this batch | reused from before | you | external or automatic

Goal: Codex running inside a fresh repository with ground rules it must follow, and a truthful list of which Codex features your environment really has.

What you do

1. YOU	Create an empty private GitHub repo called learning-os and connect it to Codex (cloud task, desktop app or CLI - whichever you will present with).
2. YOU	CLI route only: run npm i -g @openai/codex, then run codex inside the repo folder and sign in.
3. YOU	Paste Prompt A. Read the plan Codex shows. Approve it only when it matches the prompt.
4. CODEX	Creates AGENTS.md, docs/capabilities.md and the folder layout.
5. YOU	Open docs/capabilities.md. Next to each of the nine features write native, partial or fallback, based on what you can actually see in your environment.
6. YOU	Merge Codex's pull request, run git pull, then git tag batch-0-done.

Prompt A - paste this into Codex

```
I am building Learning OS for a Codex hackathon. Do not write app code yet. 1) Create AGENTS.md at the repo root with these rules: work in small steps; show a plan before writing code; run tests before saying done; never claim a feature works unless I have seen it run; cite the official doc page for any claim about Codex features. 2) Create docs/capabilities.md: a table of nine Codex features (subagents, git worktrees, MCP, skills, hooks, scheduled tasks, memory, image input, voice) with columns: available in this environment (yes, no or unsure), how to use it here, official doc link. Where unsure, say unsure and tell me how I can check. 3) Create the folders app/, tests/, fixtures/, docs/, .agents/skills/ and .codex/agents/, each with a .gitkeep file. Show me the plan first and wait for my approval.
```

What you should see

A plan appears in the chat first. After you approve, a pull request adds AGENTS.md, docs/capabilities.md with nine rows, and the empty folders.

Done when

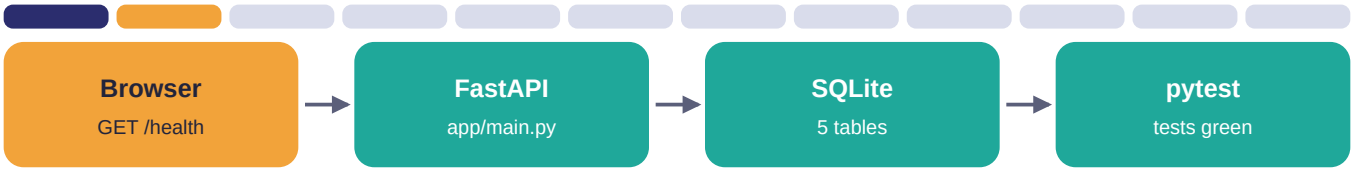
- [ ] AGENTS.md exists and lists the five rules
- [ ] capabilities.md has all nine rows, each with your own native, partial or fallback mark
- [ ] Merged, pulled and tagged batch-0-done

Capture for your presentation: Screenshot of docs/capabilities.md. It becomes your honesty slide: you only claim what is marked native.

1 Foundation: app and data model

Codex feature on show: Plan first, tests first

YOU ARE HERE: BATCH 1 OF 11



The skeleton every later batch plugs into.

new in this batch | reused from before | you | external or automatic

Goal: A running FastAPI app with a database that can hold Collections, Materials, titled Chunks, Concepts and Progress.

What you do

1. YOU	Paste Prompt B. Approve the plan.
2. CODEX	Creates app/main.py, app/models.py, requirements.txt and tests, then opens a pull request.
3. YOU	Merge, git pull, then run pip install -r requirements.txt and pytest. Read the output.
4. YOU	Run uvicorn app.main:app --reload and open http://localhost:8000/health in your browser.
5. YOU	Open app/models.py and find the five tables. If one is missing, tell Codex.
6. YOU	Tag batch-1-done.

Prompt B - paste this into Codex

```
Set up the Learning OS backend. Stack: Python 3.12, FastAPI, SQLite through SQLAlchemy, pytest. Create these tables: Collection (id, name, drive_folder_id nullable), Material (id, collection_id, title, source_type, position), Chunk (id, material_id, title, position, page_ref, selected bool default false), Concept (id, chunk_id, text, citation, status), ProgressEvent (id, concept_id, ts, confidence, correct, note). Add GET /health returning {"status":"ok"}. Add tests that create a Collection with a Material and Chunks and read them back in position order. Plan first, wait for approval, then run the tests and show me the output.
```

What you should see

pytest prints a line such as 3 passed. The browser shows {"status":"ok"}.

Done when

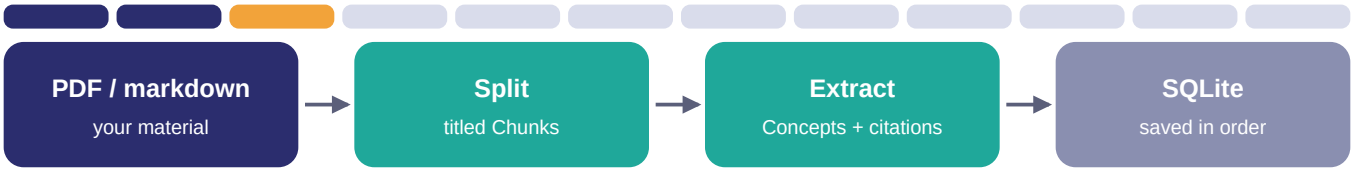
- [ ] pytest is green on your machine
- [ ] /health works in your browser
- [ ] You found all five tables in app/models.py

Capture for your presentation: Screenshot of the passing tests and of the /health page.

2 Ingestion: titled chunks with citations

Codex feature on show: A Codex cloud task you review as a pull request; model calls stubbed so tests run offline

YOU ARE HERE: BATCH 2 OF 11



Same pipeline will take Drive files, links and photos in later batches.

new in this batch | reused from before | you | external or automatic

Goal: Two chemistry chapters become titled sections, and every section becomes short testable concepts that each point back to their source page.

What you do

1. YOU	Put sample material in fixtures/: two short chemistry chapters you have the right to use (or ask Codex to write two short original ones, Chapter 3 and Chapter 4, as PDFs), plus one short markdown note.
2. YOU	Paste Prompt C. Approve the plan.
3. CODEX	Builds app/ingest.py and app/llm.py (the one place model calls live), with tests that use a stub.
4. YOU	Set your key in your terminal: export OPENAI_API_KEY=your-key. Never commit it.
5. YOU	Run python -m app.ingest fixtures/chem_ch3.pdf --collection 1 and read the printed tree.
6. YOU	Open the PDF and check three citations by hand. Does each point to the right page?
7. YOU	Repeat for Chapter 4 and the markdown note. Run pytest. Tag batch-2-done.

Prompt C - paste this into Codex

```
Build ingestion v1 for PDF and markdown only. For each file: 1) split it into titled sections (use
headings; if there are none, ask the model to propose section titles) and store them as Chunks in
order with page_ref; 2) from each Chunk extract 3 to 8 discrete concepts, each one testable idea in
plain language, each with a citation such as 'chem_ch3.pdf p.4'. All model calls go through app/llm.py
using the OpenAI API with the key from the OPENAI_API_KEY environment variable. Add a CLI: python -m
app.ingest <file> --collection <id> that prints each Chunk with its Concepts and citations. Add tests
that stub the model call. Plan first.
```

What you should see

A printed tree: each chapter, its titled sections in order, and under each section 3 to 8 short concepts ending in a citation like [chem_ch3.pdf p.4].

Done when

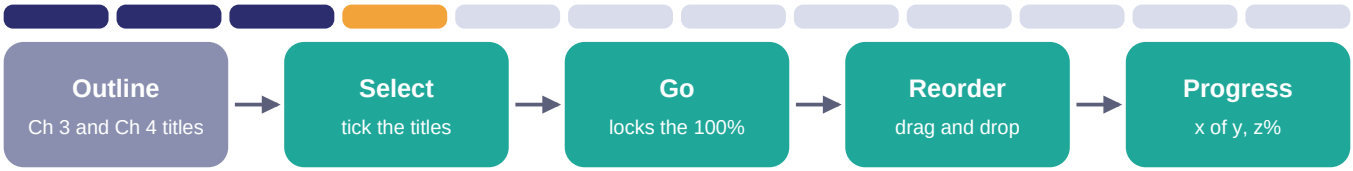
- [ ] Both chapters give titled sections in the right order
- [ ] Three hand-checked citations are correct
- [ ] pytest passes with no internet access

Capture for your presentation: Screenshot of the printed tree for Chapter 3, next to the PDF page one concept cites.

3 Scope and progress: choose, drag, track

Codex feature on show: Building and judging a visual interface in one batch

YOU ARE HERE: BATCH 3 OF 11



Your selection, not the upload, defines what counts as finished.

new in this batch | reused from before | you | external or automatic

Goal: You choose exactly which titles to learn, across materials. Those become the 100 percent, you can reorder by drag and drop, and a progress bar tracks completion against your own scope.

What you do

1. YOU	Paste Prompt D. Approve the plan.
2. CODEX	Adds the outline, scope, reorder and progress endpoints and a single web page.
3. YOU	Merge, git pull, run the app, open http://localhost:8000. Pick the Collection. Chapter 3 and Chapter 4 titles appear side by side.
4. YOU	Tick five titles across both chapters and press Go. The bar must read 0 of 5 (0%).
5. YOU	Drag Chapter 4 above Chapter 3. Reload the page. The order must stay.
6. YOU	Press the temporary debug button on one title. The bar must read 1 of 5 (20%).
7. YOU	Run pytest. Tag batch-3-done.

Prompt D - paste this into Codex

```
Build the scope and progress layer. API: GET /collections/{id}/outline returns materials and chunks in order with selected flags; POST /collections/{id}/scope takes a list of chunk ids, marks them selected and locks them as the scope; PATCH /materials/reorder and PATCH /chunks/reorder persist new positions; GET /collections/{id}/progress returns completed_count, total_in_scope and percent. Front end: one HTML page with vanilla JavaScript, SortableJS drag and drop for materials and for chunks inside a material, checkboxes on chunks, a Go button that locks the scope, and a progress bar reading 'x of y (z%)'. Add a temporary debug button on each chunk to mark it complete. Tests for the progress math. Plan first.
```

What you should see

Both chapters listed with a checkbox on every title. After Go the bar reads 0 of 5. After one debug completion it reads 1 of 5 (20%). The dragged order survives a reload.

Done when

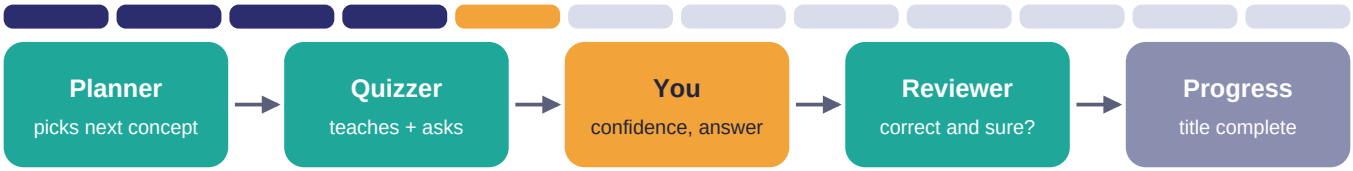
- [ ] Titles from two materials appear together
- [ ] Go sets the denominator to your selection
- [ ] Drag order persists after reload
- [ ] Progress math tests pass

Capture for your presentation: Screen recording: select, Go, drag, reload, progress. This is your strongest demo moment for the scope idea.

4 Teach and check, with subagents

Codex feature on show: Subagents: Planner, Quizzer and Reviewer

YOU ARE HERE: BATCH 4 OF 11



The Reviewer owns the word done, using confidence against accuracy, without over-testing.

new in this batch | reused from before | you | external or automatic

Goal: One concept at a time: it is taught from your source, you rate your confidence and answer, and the Reviewer judges whether the title is complete.

What you do

1. YOU	Paste Prompt E1. Approve.
2. CODEX	Creates .codex/agents/planner.toml, quizzer.toml and reviewer.toml.
3. YOU	Open each file. One job and a short description each.
4. YOU	Ask Codex how the running app can call these agents (for example non-interactive Codex or the Codex SDK), with the doc page, and record it in capabilities.md. If it cannot, the app reuses the same prompts through app/llm.py and you say so in the demo.
5. YOU	Paste Prompt E2. Approve.
6. YOU	Start a session on your scoped titles. Give a confidence from 1 to 5 before each answer. Answer one well and one badly on purpose. Watch the verdicts and the bar. Run pytest. Tag batch-4-done.

Prompt E1 - paste this into Codex

```
Create three Codex custom agents as TOML files in .codex/agents/: planner (picks the next concept from the selected scope, earliest position first, weakest first), quizzer (teaches one concept from its source text with the citation, then asks one short check question, never revealing the answer first), reviewer (after each check, compares the learner's confidence 1 to 5 with correctness, records a ProgressEvent, and decides whether the title is complete: correct and confident, no sign of guessing, and not over-tested). Use the current Codex docs for the exact file format and name the page you used.
```

Prompt E2 - paste this into Codex

```
Build the teach-and-check loop as an API and a UI panel: POST /sessions starts a session on the locked scope; GET /sessions/{id}/next returns the teaching text, citation and check question for one concept; POST /sessions/{id}/answer takes the answer and confidence and returns feedback and the Reviewer verdict. Completion of a title is decided by the Reviewer logic, not a fixed count. Tests with stubbed model calls. Plan first.
```

What you should see

Per concept: a short cited explanation, then one check question. After your answer, a two-line verdict on your confidence, the result and whether the title is complete.

Done when

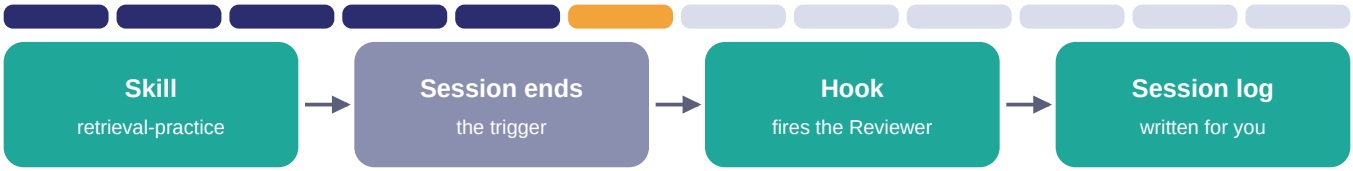
- [ ] Three agent files, one job each
- [ ] A confident wrong answer was flagged, not rewarded
- [ ] A completed title moves the progress bar

Capture for your presentation: Screenshot of one verdict (confidence versus accuracy) and the three agent files.

5 A reusable skill and an automatic hook

Codex feature on show: Skills and hooks

YOU ARE HERE: BATCH 5 OF 11



You do nothing at the end of a session; the hook does it.

new in this batch | reused from before | you | external or automatic

Goal: Retrieval-practice question writing becomes a reusable Codex skill, and the Reviewer runs automatically when a session ends.

What you do

1. YOU	Paste Prompt F1 (the skill). Approve.
2. YOU	Open .agents/skills/retrieval-practice/SKILL.md. It needs a name, a description and clear steps. Then say: use the retrieval-practice skill on the first title in Chapter 3. Check the questions mix recall, explain-it-back and apply, each cited.
3. YOU	Ask Codex which hook events your version supports and how to switch hooks on, with the doc page. Record it in capabilities.md.
4. YOU	Paste Prompt F2 (the hook). Approve.
5. YOU	Finish a study session. docs/session-log.md must gain a new entry without you running anything.
6. YOU	Tag batch-5-done.

Prompt F1 - paste this into Codex

```
Create a Codex skill at .agents/skills/retrieval-practice/SKILL.md. Frontmatter: name
retrieval-practice and a one-line description saying when to use it. Body: given a title and its
concepts, write 5 questions that mix recall, explain-it-back and apply; never put the answer in the
question; every question carries the citation of its concept; return JSON with question,
expected_answer and citation. Show me the file and one sample run.
```

Prompt F2 - paste this into Codex

```
Add a hook that runs the Reviewer summary automatically when a study session ends and appends a dated
entry to docs/session-log.md with titles covered, confidence versus accuracy and titles completed.
First check the current Codex docs for supported hook events and how to enable hooks, and tell me
which event you chose and why. If no Codex event matches 'study session ended', say so, make the app
call the same script at session end instead, and label it honestly in capabilities.md. Plan first.
```

What you should see

The sample run returns five cited questions. After a session, a dated entry appears in docs/session-log.md on its own.

Done when

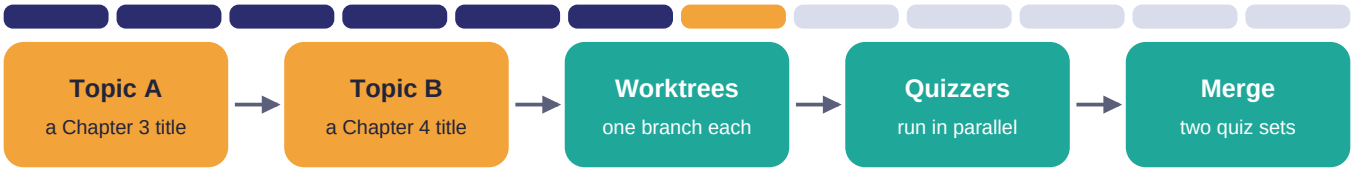
- [ ] Skill file has name, description and steps
- [ ] Sample questions all carry citations
- [ ] A session-log entry appeared without a manual command
- [ ] capabilities.md states how the hook is wired

Capture for your presentation: Screenshot of the skill file and of a session-log entry that appeared by itself.

6 Parallel quizzers in git worktrees

Codex feature on show: Git worktrees with parallel subagents

YOU ARE HERE: BATCH 6 OF 11



Isolation is the point: neither quizzer can disturb the other's files.

new in this batch | reused from before | you | external or automatic

Goal: Two topics are quizzed at the same time, each in its own isolated worktree, then merged cleanly.

What you do

1. YOU	Pick two titles from different chapters.
2. YOU	Paste Prompt G. Approve the plan.
3. YOU	While it runs, open a second terminal and run git worktree list. You should see two extra folders.
4. YOU	When it finishes, read both quiz sets. Each question must cite its own chapter, with nothing leaked from the other.
5. YOU	Run pytest. Tag batch-6-done.

Prompt G - paste this into Codex

```
Generate retrieval-practice sets for two titles in parallel. Use two quizzer subagents, each working
in its own git worktree on its own branch (quiz/topic-a and quiz/topic-b), each using the
retrieval-practice skill and writing to data/quiz_sets/<topic>.json. When both finish, merge both
branches into main and tell me about any conflict. First check the current Codex docs for how
worktrees and parallel subagents are started in my environment and tell me the steps you will use.
Plan first.
```

What you should see

While the run is active, git worktree list shows main plus two more folders. Afterwards two JSON files exist in data/quiz_sets/.

Done when

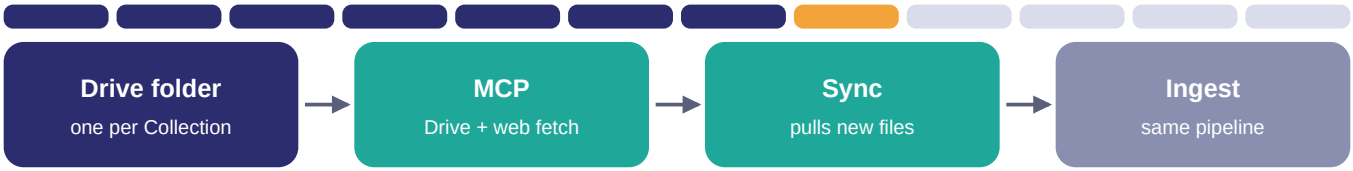
- [ ] Two worktrees were visible during the run
- [ ] Two quiz sets cite their own chapters
- [ ] Merge finished and tests pass

Capture for your presentation: Screenshot of git worktree list during the run and the two files after the merge.

7 MCP: Drive folder, web page and YouTube

Codex feature on show: MCP connections

YOU ARE HERE: BATCH 7 OF 11



Only Drive, public web pages and YouTube captions are built. Nothing else is claimed.

new in this batch | reused from before | you | external or automatic

Goal: A Collection is tied to a Google Drive folder. New files, public web pages and YouTube captions flow into the same pipeline.

What you do

1. YOU	Run codex mcp list in your terminal to see what is connected.
2. YOU	Ask Codex how to add a Google Drive MCP server to your setup, with the exact command and doc page. Run the command yourself and sign in when asked.
3. YOU	In Drive create a folder named Learning OS - Chemistry and drop one PDF in it.
4. YOU	Paste Prompt H. Approve.
5. YOU	Call the sync, reload the outline. The Drive file appears as a material with titled chunks.
6. YOU	Paste a public article link, then a YouTube link that has captions. Citations must show a paragraph reference or a timestamp.
7. YOU	Try a YouTube link with no captions. You must get a friendly message, not a crash. Tag batch-7-done.

Prompt H - paste this into Codex

```
Connect a Collection to a Google Drive folder through MCP. Store drive_folder_id on the Collection.
Add POST /collections/{id}/sync that lists files in that folder through the Drive MCP, ingests files
not seen before with the existing pipeline, and returns what it added. Add POST /collections/{id}/link
for a public web page or YouTube link: fetch web page text through MCP or a fetch tool; for YouTube
fetch captions only and cite by timestamp; if there are no captions, return a clear message. Do not
attempt paywalled or login-only sites. Plan first and tell me which MCP tools you will call.
```

What you should see

After sync, the Drive file shows up in the outline. Links produce chunks with web or timestamp citations.

Done when

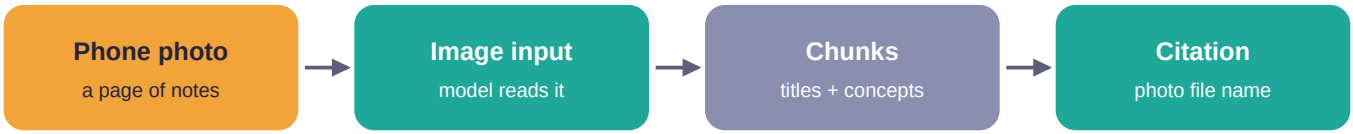
- [ ] One Drive file ingested after sync
- [ ] One web page and one captioned YouTube link ingested
- [ ] No-captions case shows a friendly message

Capture for your presentation: Screenshot of the Drive folder beside the new material. In your talk say: gated content such as Udemy is out of scope; the design is extensible through MCP.

8 Photo notes

Codex feature on show: Image input

YOU ARE HERE: BATCH 8 OF 11



Reads clear photos well; unreadable ones ask for a retake instead of guessing.

new in this batch | reused from before | you | external or automatic

Goal: A phone photo of a notes page becomes titled sections and cited concepts like any other material.

What you do

1. YOU	Take a clear phone photo of a printed or handwritten notes page. Save it as fixtures/notes_photo.jpg.
2. YOU	Take one deliberately blurry photo and save it as fixtures/blurry.jpg.
3. YOU	Paste Prompt I. Approve.
4. YOU	Run python -m app.ingest fixtures/notes_photo.jpg --collection 1 and compare the extracted text with the real page. If it is far off, ask Codex to improve the prompt.
5. YOU	Run the blurry photo. You must get a retake message.
6. YOU	Run pytest. Tag batch-8-done.

Prompt I - paste this into Codex

```
Add image ingestion for jpg and png. Send the image to a vision-capable model through app/llm.py,
extract the text and structure, split it into titled sections and extract concepts exactly as for
PDFs. Cite as 'photo <filename>' plus section title. If the image is unreadable, return a clear
message asking for a retake. Tests with a stub. Plan first.
```

What you should see

The clear photo prints titled sections and cited concepts. The blurry one asks for a retake.

Done when

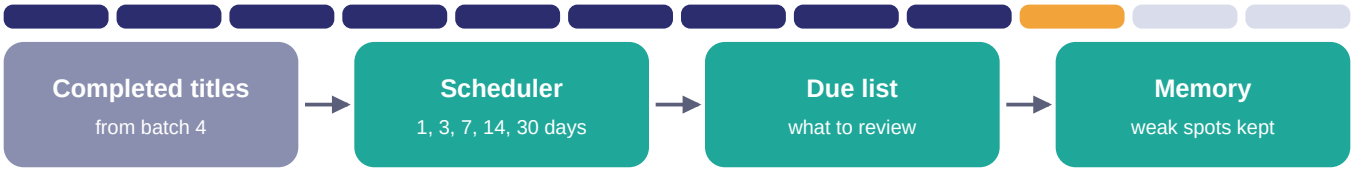
- [ ] Clear photo ingested with sensible titles
- [ ] Blurry photo is refused politely
- [ ] Tests pass

Capture for your presentation: Side by side: your photo, and the concepts extracted from it.

9 Spaced review and memory

Codex feature on show: Scheduled tasks and memory

YOU ARE HERE: BATCH 9 OF 11



This is the spacing effect, run by a schedule instead of your willpower.

new in this batch | reused from before | you | external or automatic

Goal: Completed titles come back at growing intervals, a failed review resets the ladder, and weak spots persist across sessions.

What you do

1. YOU	Paste Prompt J. Approve.
2. YOU	Run python -m app.review --today +0d. Nothing should be due.
3. YOU	Run it with --today +3d. Titles completed earlier must now be due.
4. YOU	Fail a review on purpose. That title must drop back to the shortest interval.
5. YOU	Ask Codex how to create a recurring scheduled task in your environment that runs scripts/daily_review.py every morning, with the schedule rule and doc page. Create it where Codex tells you.
6. YOU	Ask Codex what its memory stores in your environment and whether the app can write calibration history to it. Record the answer in capabilities.md. Your database keeps the history either way.
7. YOU	Tag batch-9-done.

Prompt J - paste this into Codex

```
Build spaced review. When a title is completed, set next_review_at using intervals of 1, 3, 7, 14 and
30 days for each successful review; a failed review resets to 1 day. Add GET /review/due and a CLI
python -m app.review --today +Nd that fakes the date for demos. Keep confidence-versus-accuracy
history per concept and a per-Collection weak-spots list in the database. Add scripts/daily_review.py
that prints today's due titles, ready to be run by a scheduled task. Tests for the interval ladder.
Plan first.
```

What you should see

+0d: nothing due. +3d: earlier completed titles listed. A failed review shows next review tomorrow.

Done when

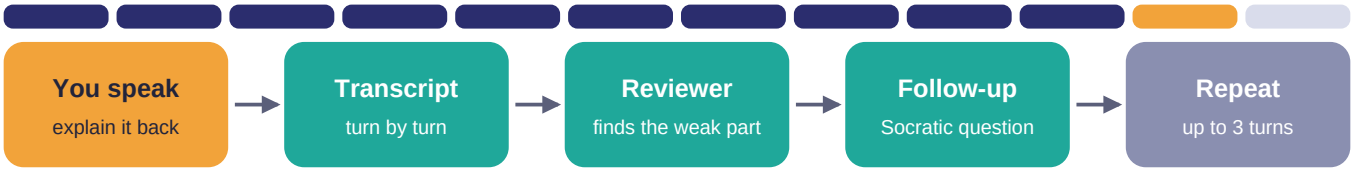
- [ ] Date jump shows due titles
- [ ] A failed review resets the ladder
- [ ] A scheduled task exists in your environment
- [ ] Memory answer recorded honestly

Capture for your presentation: Terminal screenshots for +0d and +3d, plus the scheduled task in your Codex settings.

10 Voice teach-back

Codex feature on show: Voice

YOU ARE HERE: BATCH 10 OF 11



Turn-based conversation, not overlapping real-time speech.

new in this batch | reused from before | you | external or automatic

Goal: You explain a concept aloud, the Reviewer finds the weak part and asks a Socratic follow-up, for up to three turns.

What you do

1. YOU	Reread the voice row in capabilities.md. Settle with Codex how voice works for the app: Codex voice if available, otherwise browser speech recognition and speech output.
2. YOU	Paste Prompt K. Approve.
3. YOU	Open a session in Chrome. Explain a concept aloud and leave out one key part on purpose.
4. YOU	Check the follow-up targets the missing part, not a generic question.
5. YOU	Complete three turns. Check the transcript is saved and counts toward the verdict.
6. YOU	Tag batch-10-done.

Prompt K - paste this into Codex

```
Add a voice teach-back mode to the session panel. Flow: the learner presses speak and explains the concept aloud; show the transcript; the Reviewer compares it with the concept text, finds the weakest missing or wrong part and asks one Socratic follow-up, spoken and shown; the learner answers; repeat up to 3 turns; then give the verdict. Use browser speech recognition and speech synthesis unless docs/capabilities.md says Codex voice is available to the app. Always offer typing as a fallback. Save the transcript with the ProgressEvent. State in the UI help text that this is turn-based. Plan first.
```

What you should see

Your spoken explanation appears as text, then a pointed follow-up about the part you skipped.

Done when

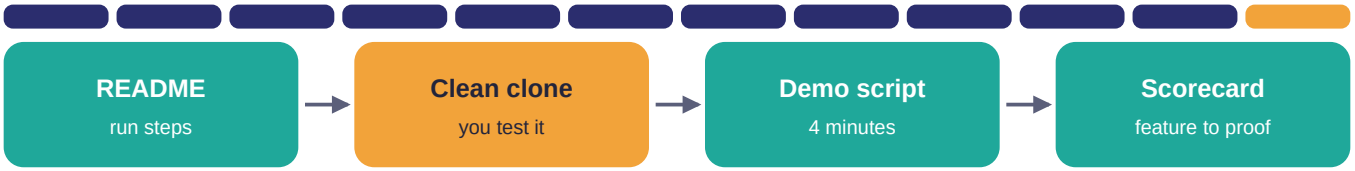
- [ ] Follow-up targets the gap you left
- [ ] Three turns complete
- [ ] Typing fallback works

Capture for your presentation: Screen recording with sound of a three-turn voice exchange.

11 Polish and demo

Codex feature on show: Everything, shown honestly

YOU ARE HERE: BATCH 11 OF 11



The last check is the one a judge would do: clone it and run it.

new in this batch | reused from before | you | external or automatic

Goal: A stranger can run the project from the README, and you can present it in four minutes without claiming anything that is not working.

What you do

1. YOU	Paste Prompt L. Approve.
2. YOU	Clone the repo fresh into a new folder and follow the README exactly, with no help. If anything fails, tell Codex to fix the README or the code and repeat.
3. YOU	Rehearse the demo script with a timer.
4. YOU	Fill in the scorecard on the last page of this playbook.
5. YOU	Record your demo. Tag batch-11-done, then submit.

Prompt L - paste this into Codex

```
Write README.md: what Learning OS is in three sentences; a flow diagram in mermaid; numbered setup
steps from a fresh clone to a running app; how to run tests; how each Codex feature is used, with
links to the files (AGENTS.md, .codex/agents, .agents/skills, hooks, the scheduled task, worktrees,
MCP config), each marked native, partial or fallback exactly as in docs/capabilities.md; an Out of
scope section (gated third-party courses, video file upload); and a four minute demo script in this
order: Collection, ingest Chapters 3 and 4, select titles, Go and drag, teach and check with
confidence, voice follow-up, spaced review with the date jump, scorecard. Do not claim anything that
is not working. Plan first.
```

What you should see

A README a stranger can follow, and a demo you can finish inside four minutes.

Done when

- [ ] A fresh clone runs from the README alone
- [ ] Demo rehearsed inside four minutes
- [ ] Every claim matches capabilities.md
- [ ] Scorecard filled in

Capture for your presentation: Your final recording and the README.

iii Final scorecard and recovery

Fill this in before you present. Mark each feature native, partial or fallback exactly as in docs/capabilities.md, and attach the proof you captured.

Codex feature	Batch	Proof to show	Mark
Subagents	4, 6	Three agent files and a verdict	_____
Git worktrees	6	git worktree list during the run	_____
MCP	7	Drive folder beside the new material	_____
Skills	5	SKILL.md and a sample run	_____
Hooks	5	Session-log entry that appeared alone	_____
Scheduled tasks	9	Task in settings plus +3d output	_____
Memory	9	Weak-spots list that persists	_____
Image input	8	Photo beside extracted concepts	_____
Voice	10	Recording of a three-turn exchange	_____

If something breaks

- Copy the exact error into Codex and ask: explain the cause first, then propose a fix, and wait for my approval.
- If the fix grows beyond the batch, stop. Reset to the last tag and rerun the batch prompt with the specific problem added.
- If a Codex feature is not available in your environment, mark it fallback in capabilities.md, build the honest alternative, and say so in the demo. That is a better answer than a claim you cannot show.